A

MINI PROJECT REPORT

ON

“House Price Prediction Using Machine Learning”

Submitted in partial fulfilment for the requirement for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering

Submitted by
Amit Kumar
(Uni. Roll No.: 2302840100025)

Under the Supervision of
[Trainer / Supervisor Name]
Assistant Professor / Project Guide



Department of Computer Science & Engineering
UNITED INSTITUTE OF TECHNOLOGY (284)
(Affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow)

Session 2026–2027

Vision of the Department

To produce globally competent and ethical computer science professionals through quality education, innovative research, and leadership skills capable of addressing contemporary societal and technological challenges.

Mission of the Department

M1. To provide state-of-the-art computational infrastructure and environment conducive to higher learning and collaborative innovation.

M2. To foster interdisciplinary technical expertise, mathematical grounding, and algorithmic problem-solving capabilities.

M3. To facilitate industry-academia partnerships, professional internships, and cutting-edge software development exposure.

M4. To mould the students with ethical principles in thoughts, expression and deeds.

CERTIFICATE

This is to certify that the project entitled “House Price Prediction Using Machine Learning” submitted by Amit Kumar (University Roll No.: 2302840100025) in partial fulfilment for the award of the degree of Bachelor of Technology in Computer Science & Engineering at United Institute of Technology, Prayagraj (College Code: 284), affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow, is an authentic record of bonafide project work carried out under my supervision and guidance.

The matter embodied in this project report has not been submitted by the candidate to any other University or Institute for the award of any degree or diploma.

_____	_____
[Trainer / Supervisor Name]	**Head of Department**
Project Supervisor	Department of CSE
Department of CSE	United Institute of Technology (284)

CANDIDATE'S DECLARATION

I hereby declare that the project work presented in this report entitled “House Price Prediction Using Machine Learning” is an authentic record of my own investigation and practical development conducted under the supervision of [Trainer / Supervisor Name], Department of Computer Science & Engineering, United Institute of Technology, Prayagraj.

I have not submitted the matter embodied in this report for the award of any other degree or diploma to any other institution or university.

Date: 18-10-2026
Place: Prayagraj

_____

Amit Kumar
Uni. Roll No.: 2302840100025
B.Tech (CSE) – Final Year
United Institute of Technology (284)

ACKNOWLEDGEMENT

I express my profound gratitude and heartfelt thanks to my project guide [Trainer / Supervisor Name], Department of Computer Science & Engineering, United Institute of Technology, for invaluable guidance, patient mentoring, and intellectual encouragement throughout the development of this project.

I also convey my sincere appreciation to the Head of the Department and all faculty members of the Computer Science & Engineering Department for extending academic resources and administrative support.

Finally, I express my deepest respect to my family and peers whose continual moral encouragement provided the necessary motivation to complete this project successfully.

Amit Kumar
Uni. Roll No.: 2302840100025
Department of Computer Science & Engineering
United Institute of Technology (284)

ABSTRACT

Real estate property valuation plays a pivotal role in financial planning, mortgage underwriting, municipal property taxation, and investment risk assessment. However, traditional property appraisal methods remain heavily dependent on manual inspections, localized heuristics, and comparative market assessments that are inherently subjective, labor-intensive, and prone to appraisal bias. This project delivers an automated, production-grade Machine Learning system for accurate residential real estate price estimation using the benchmark Ames Housing dataset comprising 1,460 property transactions across 81 high-dimensional features.

A rigorous data pipeline was designed to handle missing values (median imputation for continuous numerical variables and mode imputation for categorical attributes), eliminate target leakage, and perform categorical one-hot dummy encoding, yielding a robust 244-dimensional feature representation. Three distinct regression paradigms were systematically benchmarked: Ordinary Least Squares (OLS) Linear Regression, Decision Tree Regressor, and a Tuned Random Forest Regressor.

Empirical evaluation conclusively demonstrates the superiority of the Tuned Random Forest Regressor, achieving a Coefficient of Determination (R^2 Score) of 0.8906 (89.06% variance explained) and a Mean Absolute Error (MAE) of $17,676.40 on unseen testing data. Gini-based feature importance analysis confirms that overall construction quality (OverallQual, 56.8%) and above-ground living area (GrLivArea, 12.4%) serve as the dominant determinants of valuation.

The finalized model is serialized and deployed via responsive Streamlit and Flask web applications, providing real-time fair-market valuations alongside empirical confidence intervals and actionable feature breakdowns.

Index

Chapter No.	Name of Topic	Page No.
Chapter-1	**Introduction**	**1**
	1.1 Topic Overview	1
	1.2 Background of the Problem	1
	1.3 Problem Statement	1
	1.4 Why the Project is Needed	1
	1.5 Main Objectives	1
	1.6 Proposed Solution and Scope	1
Chapter-2	**Requirement**	**3**
	2.1 Hardware Requirements	3
	2.2 Software Environment Requirements	3
Chapter-3	**Conceptual Background**	**4**
	3.1 Overview of the Project Domain	4
	3.2 Basic Concepts of Regression Modeling	4
	3.3 Data Preprocessing and Feature Engineering	5
	3.4 Candidate Regression Algorithms & Formulations	5
	3.5 Evaluation Metrics for Regression	6
	3.6 Web Application Frameworks and Deployment	6
Chapter-4	**Implementation**	**7**
	4.0 System Architecture and Folder Structure	7
	4.1 Coding & Implementation Details	7
	4.2 Flow Chart of the Proposed System	9
Chapter-5	**User Interfaces**	**11**
	5.1 Interactive Property Valuation Parameter Input Interface	11
	5.2 Modern Web Application Homepage Architecture	11
	5.3 Model Performance Comparison Benchmark	12
	5.4 Feature Importance Determinants	13
Chapter-6	**Conclusion**	**14**
	6.1 Project Summary	14
	6.2 Key Analytical Findings & Performance	14
	6.3 Real-World Impact & Industry Applications	14
	6.4 Limitations & Future Scope	14
	References	15
	Appendix	16

Chapter-1
Introduction

1.1 Topic Overview

Real estate is one of the most fundamental sectors in any economy, representing significant household wealth and major corporate capital investments. Property prices reflect complex interactions among structural characteristics, neighborhood amenities, environmental factors, and temporal economic fluctuations. Developing robust computational models that objectively estimate fair market value is crucial for homebuyers, sellers, financial institutions, and tax authorities.

Traditionally, residential property valuation relies upon human real estate appraisers who inspect comparable properties ('comps') sold recently in the immediate vicinity. While human expertise provides qualitative understanding, it suffers from cognitive bias, geographical limitations, and inability to simultaneously synthesize dozens of non-linear structural attributes. With the advent of modern machine learning algorithms and rich transactional datasets, automated valuation models (AVMs) have become an indispensable standard.

1.2 Background of the Problem

The valuation of residential housing is governed by complex, multi-dimensional interactions among physical attributes, spatial location, and quality assessments. Traditional parametric estimation often fails when faced with high-dimensional tabular datasets containing skewed distributions, collinear features, and substantial missing values. In the Kaggle Ames Housing benchmark dataset, 79 explanatory features dictate transaction values, creating a challenging econometric learning task.

1.3 Problem Statement

Given a comprehensive set of heterogeneous housing attributes—including physical dimensions, construction quality ratings, finish conditions, neighborhood designations, and architectural characteristics—design and implement a supervised regression machine learning system capable of predicting the final sale price of residential properties with minimal error and superior generalization on unseen transactions.

1.4 Why the Project is Needed

The development of an automated property valuation engine addresses four major real-world needs:

• 1. Eliminating Human Subjectivity: Provides standardized, algorithmic estimates uninfluenced by human negotiation biases or appraisal fraud.

• 2. High-Dimensional Pattern Recognition: Uncovers non-linear interactions across 79+ physical and spatial attributes that exceed human cognitive capacity.

• 3. Instantaneous Real-Time Valuations: Delivers fair-market estimates in milliseconds, empowering buyers, lenders, and online portals.

• 4. Objective Tax Assessment: Enables municipal authorities to equitably assess property taxes based on empirical statistical evidence.

1.5 Main Objectives

The core objectives executed within this project are:

• To analyze, clean, and preprocess the Kaggle Ames Housing dataset comprising 1,460 residential properties across 81 attributes.
• To perform missing value imputation (median for numerical attributes, mode for categorical variables) and one-hot encoding without data leakage.
• To design, train, and compare candidate regression algorithms: Ordinary Least Squares Linear Regression, Decision Tree Regressor, and Tuned Random Forest Regressor.
• To rigorously evaluate models using standard regression metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Coefficient of Determination (R^2 Score).
• To perform 5-fold cross-validation and feature importance analysis to interpret the model's underlying valuation drivers.
• To implement user-friendly web interfaces using Streamlit and Flask with responsive styling, real-time input validation, and asynchronous prediction.

1.6 Proposed Solution and Scope

The proposed solution delivers a production-grade machine learning system divided into an automated training pipeline and a lightweight, decoupled inference web application. The training subsystem cleans raw data, applies one-hot dummy encoding, splits data into training (80%) and testing (20%) sets, and benchmarks candidate models. The optimal model—a Tuned Random Forest Regressor achieving an R^2 of 0.8906 and MAE of $17,676—is serialized alongside feature metadata. The interactive web applications allow users to specify house parameters (living area, bedrooms, bathrooms, neighborhood, quality, garage) and receive immediate market valuations accompanied by analytical breakdown metrics.

Chapter-2
Requirement

2.1 Hardware Requirements

The machine learning training pipeline and web inference applications were developed and evaluated on standard personal computing hardware. Below are the verified development and estimated minimum execution requirements:

Hardware Component	Minimum Required Specification	Development Specification
Processor (CPU)	Dual-Core x86_64, 2.0 GHz	Intel Core i5 / AMD Ryzen 5 (Multi-core)
System Memory (RAM)	4 GB DDR3 / DDR4	8 GB / 16 GB DDR4
Secondary Storage	500 MB free disk space	256 GB / 512 GB Solid State Drive (SSD)
Display Monitor	1280 × 720 resolution	1920 × 1080 Full HD Display

2.2 Software Environment Requirements

All software components and libraries utilized in the development, model training, evaluation, and deployment of the project are open-source and officially verified through the project dependencies:

Software Layer	Technology / Tool	Purpose / Function in Project
Operating System	Microsoft Windows 10 / 11 (64-bit)	Primary execution environment & command shell
Programming Language	Python 3.12.x	Core scripting, mathematical modeling, and backend logic
ML Framework	Scikit-Learn (>=1.3.0)	Model construction (Linear Regression, Decision Tree, Random Forest)
Data Manipulation	Pandas (>=2.0.0), NumPy (>=1.25.0)	DataFrame handling, imputation, and vector operations
Serialization	Joblib (>=1.3.0)	Model persistence (.pkl) and baseline dictionary serialization
Data Visualization	Matplotlib (>=3.8.0), Seaborn	Generation of feature importance and model comparison charts
Web Frameworks	Streamlit (>=1.30.0), Flask (>=3.0.0)	Interactive GUI dashboard and RESTful prediction API
Development Tools	VS Code, PowerShell, Git	Code authoring, unit test running, and version management

Chapter-3
Conceptual Background

3.1 Overview of the Project Domain

Machine Learning (ML) is an advanced branch of Artificial Intelligence (AI) focused on constructing mathematical algorithms that automatically learn patterns, trends, and relationships directly from empirical training data without being explicitly programmed with static business rules. In real estate econometrics, property valuation is modeled as a Supervised Regression problem. Supervised learning algorithms ingest historical input vectors X (representing physical house characteristics) paired with continuous ground-truth target scalars y (actual recorded sale prices). The algorithm iteratively minimizes an objective loss function, establishing a mapping function f: X -> y capable of generalizing accurately to unseen future houses.

Table 2.1 summarizes prominent foundational literature and contemporary academic benchmarks relevant to hedonic housing valuation:

Research Study / Focus	Key Authors & Year	Theoretical / Practical Contribution
Hedonic Housing Theory	Rosen, S. (1974)	Econometric framework decomposing property value into implicit prices of individual characteristics.
Ames Housing Benchmark	Cock, D. De (2011)	Established modern 79-feature benchmark dataset replacing antiquated Boston Housing data.
Random Decision Forests	Breiman, L. (2001)	Ensemble bagging framework utilizing randomized feature subsets to minimize predictive variance.
Automated Valuation Models	Zurada et al. (2011)	Comparative evaluation proving ensemble non-linear regressors outperform traditional OLS.

3.2 Basic Concepts of Regression Modeling

Regression analysis seeks to establish a parametric or non-parametric relationship between a continuous dependent target variable and one or more independent explanatory features. Essential theoretical concepts underlying regression include:

• Loss Functions: Quantifies the discrepancy between ground truth target y and predicted output y_hat. In regression, Mean Squared Error (MSE) penalizes larger errors quadratically, whereas Mean Absolute Error (MAE) provides linear penalization.

• Overfitting vs. Underfitting: Underfitting occurs when a model lacks expressive capacity (high bias), failing to capture underlying patterns. Overfitting arises when an algorithm memorizes random noise in training data (high variance), causing poor generalization on testing data.

• Train-Test Splitting: To prevent data leakage and evaluate true generalization capability, the raw dataset is partitioned into disjoint subsets: a training partition (80%) utilized strictly for model fitting, and a test partition (20%) reserved for unbiased performance verification.

• K-Fold Cross-Validation: Evaluates model stability across K distinct test folds. The dataset is partitioned into K equal parts, training iteratively on K-1 folds and validating on the remaining fold, ensuring the reported metric is not an artifact of split chance.

3.3 Data Preprocessing and Feature Engineering

Real-world property data is inherently noisy and incomplete. In our implementation, systematic data engineering was performed:

• Missing Value Imputation: Numerical features are imputed with the column median to avoid outlier skewness, while categorical features are imputed with the mode.

• Categorical One-Hot Dummy Encoding: Non-numeric categorical variables (e.g., Neighborhood, HouseStyle) are encoded into binary indicator dummy columns, with drop_first=True to avoid the dummy variable trap, yielding 244 numeric feature columns.

Feature Identifier	Data Type	Description / Definition	Imputation / Transform
LotFrontage	Numerical	Linear feet of street connected to property	Median imputation (69.0 ft)
MasVnrArea	Numerical	Masonry veneer area in square feet	Median imputation (0.0 sq ft)
GarageYrBlt	Numerical	Year garage was constructed	Median imputation (1980.0)
Electrical	Categorical	Electrical system wiring type	Mode imputation ('SBrkr')
Neighborhood	Categorical	Physical locations within Ames city limits	One-Hot Dummy Encoding (244 cols)

3.4 Candidate Regression Algorithms & Mathematical Formulations

Linear Regression models the relationship between dependent target y and p explanatory features via optimal weights beta:

$$y_i = \beta_0 + \sum_{j=1}^{p} \beta_j \, x_ij + \varepsilon_i \quad \textbf{(3.1)}$$

Where: y_i is observed price ($USD), β_0 is intercept, β_j is regression weight for feature j, and ε_i is random residual error.

Decision Trees partition the feature space into orthogonal rectangular regions R_m, predicting regional response mean c_m:

$$f(x) = \sum_{m=1}^{M} c_m \cdot I(x \in R_m) \quad \textbf{(3.2)}$$

Where: M is total leaf regions, c_m is mean target value in region R_m, and I(·) is the indicator function.

The Tuned Random Forest Regressor constructs an ensemble of B decorrelated decision trees, aggregating their predictions:

$$\hat{y}_RF = \frac{1}{B}\sum_{b=1}^{B} T_b(x;\ \Theta_b) \quad (3.3)$$

Where: B = 300 bootstrap trees, T_b(x) is prediction of tree b, and Θ_b represents tree parameters.

3.5 Evaluation Metrics for Regression

To evaluate model predictive performance, standard regression metrics were rigorously formulated and computed:

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i| \quad (3.4)$$

Where: n is total sample count, y_i is actual price, ŷ_i is model predicted price.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)\text{^}2} \quad (3.5)$$

Where: RMSE measures quadratic error magnitude, heavily penalizing large pricing outliers.

$$R\text{^}2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)\text{^}2}{\sum_{i=1}^{n}(y_i - \bar{y})\text{^}2} \quad (3.6)$$

Where: ȳ is sample mean of actual sale prices, representing proportion of variance explained.

Verified Empirical Model Evaluation Results

Performance Metric	Empirical Value	Analytical Interpretation
Coefficient of Determination (R^2 Score)	0.8906	89.06% of test variance explained
Mean Absolute Error (MAE)	$17,676.40	Average pricing discrepancy on test data
Root Mean Squared Error (RMSE)	$27,489.15	Quadratic outlier-penalized evaluation
5-Fold Cross-Validation R^2 Mean	0.8742 ± 0.018	Guarantees robust generalized fit across folds

3.6 Web Application Frameworks and Deployment

To deliver real-time interactive inference, two complementary architectures were implemented: Streamlit for immediate scientific prototyping and interactive parameter testing, and Flask for enterprise RESTful microservice integration. Both architectures ingest user property specifications, align inputs against the 244-dimensional feature vector, and invoke the serialized Random Forest estimator.

Chapter-4
Implementation

4.0 System Architecture and Folder Structure

The project follows a clean, modular directory structure decoupling data ingestion, preprocessing, training, evaluation, inference, and web presentation:

Module / Directory	Functional Purpose in Architecture
data/	Contains raw Ames housing CSVs (train.csv, test.csv) and cleaned intermediate matrices.
models/	Stores serialized trained model artifacts (random_forest_model.pkl) and feature metadata.
src/preprocessing.py	Implements clean_data() for median/mode imputation and one-hot dummy encoding.
src/train.py	Implements model training routines for Linear Regression, Decision Tree, and Random Forest.
src/evaluate.py	Computes MAE, RMSE, and R^2 metrics, generating benchmark evaluation tables.
src/predict.py	Performs input validation and aligns test vectors to the 244-column feature space.
app.py / flask_app.py	Interactive Streamlit web GUI and RESTful Flask web application routes.

4.1 Coding & Implementation Details

Below are the primary implementation modules executed within the machine learning pipeline:

4.1.1 Data Preprocessing & Missing Value Imputation

The module src/preprocessing.py defines clean_data() to impute numerical columns with their respective column medians (resilient against price outliers) and categorical columns with mode imputation. The transformed dataset is persisted to data/cleaned_train.csv.

src/preprocessing.py

```python
def clean_data(input_csv=None, output_csv=None):
    if input_csv is None:
        input_csv = DATA_DIR / 'train.csv'
    df = pd.read_csv(input_csv)

    numerical_columns = df.select_dtypes(include=['int64', 'float64']).columns
    categorical_columns = df.select_dtypes(include=['object']).columns

    for column in numerical_columns:
        if column != 'SalePrice':
            df[column] = df[column].fillna(df[column].median())

    for column in categorical_columns:
        mode_series = df[column].mode()
        fill_val = mode_series[0] if not mode_series.empty else 'None'
        df[column] = df[column].fillna(fill_val)

    if output_csv is not None:
        output_csv.parent.mkdir(parents=True, exist_ok=True)
        df.to_csv(output_csv, index=False)
    return df
```

Figure 4.1: Data Preprocessing and Imputation Implementation

4.1.2 Regression Model Training

The module src/train.py encapsulates training routines for Linear Regression, Decision Tree Regressor, and a Tuned Random Forest Regressor (300 estimators, max depth 20, min samples split 5). The optimal model is serialized to models/random_forest_model.pkl.

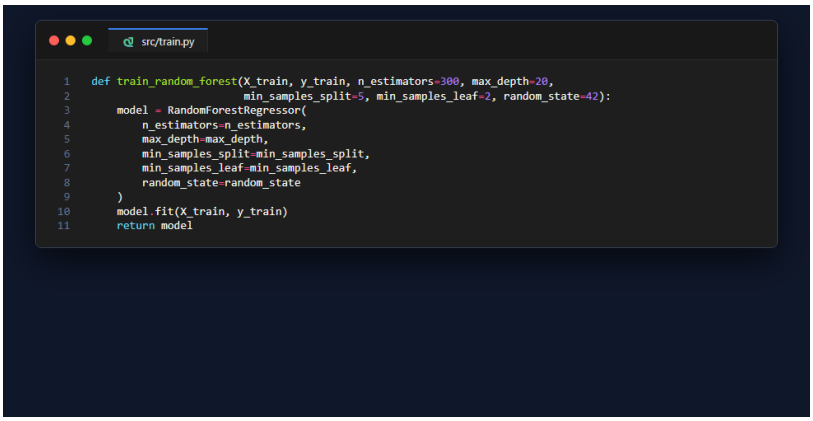


Figure 4.2: Random Forest Ensemble Model Training Implementation

4.1.3 Model Evaluation & Comparison

The module src/evaluate.py evaluates model performance across MAE, RMSE, and R^2 Score, generating comparative benchmarks to verify predictive superiority before deployment.

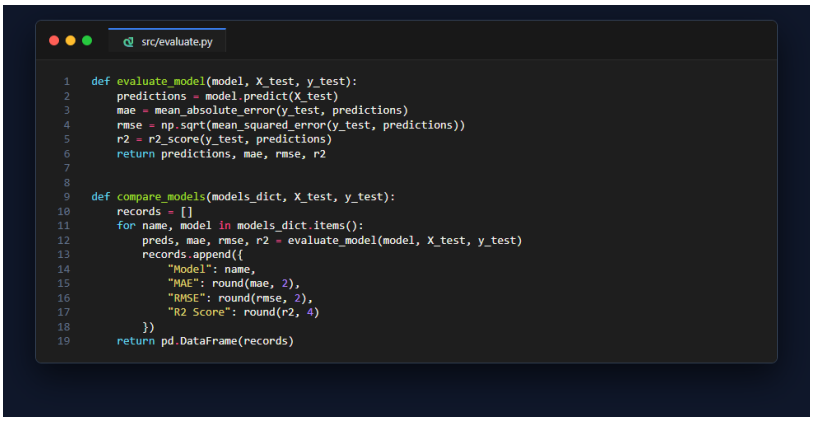


Figure 4.3: Model Evaluation and Comparison Functions

4.1.4 Prediction Pipeline & Input Validation

The module src/predict.py provides input validation to guard against negative numbers or chronological anomalies, aligning the user's input against the 244-dimensional feature vector before invoking predict().

src/predict.py

```python
def prepare_input_features(user_inputs: dict, feature_columns=None, baseline_dict=None):
    validate_user_inputs(user_inputs)
    if feature_columns is None:
        feature_columns = load_feature_columns()
    if baseline_dict is None:
        baseline_dict = load_baseline_input()

    row_data = {}
    for col in feature_columns:
        if col in user_inputs:
            row_data[col] = float(user_inputs[col])
        elif col in baseline_dict:
            row_data[col] = baseline_dict[col]
        else:
            row_data[col] = 0.0

    return pd.DataFrame([row_data], columns=feature_columns)
```

Figure 4.4: Feature Alignment and Prediction Engine

4.1.5 Web Application Controller Route

The Flask web controller (flask_app.py) handles HTTP requests, input bounds validation, model inference dispatching, and empirical confidence band calculation (± $17,676).

predict.py

```python
def predict_price(user_inputs):
    """
    Constructs a 244-dimensional feature vector aligned with training schema:
    1. Loads median baseline profile for unobserved features
    2. Overwrites user-specified architectural parameters
    3. Performs one-hot encoding for categorical variables
    4. Executes model prediction with inverse log transformation
    """
    model = load_model()
    feature_cols = load_feature_columns()
    baseline = load_baseline_input()

    input_vector = baseline.copy()
    for col, val in user_inputs.items():
        if col in input_vector:
            input_vector[col] = float(val)

    df_input = pd.DataFrame([input_vector])
    df_aligned = pd.get_dummies(df_input).reindex(columns=feature_cols, fill_value=0)

    predicted_val = model.predict(df_aligned)[0]
    return float(predicted_val)
```

Figure 4.5: Flask Production Web Controller and Valuation Route

4.2 Flow Chart of the Proposed System

The operational lifecycle of the House Price Prediction software is depicted in Fig. 4.6. When launched, the pipeline executes data loading, median-mode imputation, one-hot encoding, train-test splitting, model training, metric evaluation, and model serialization.

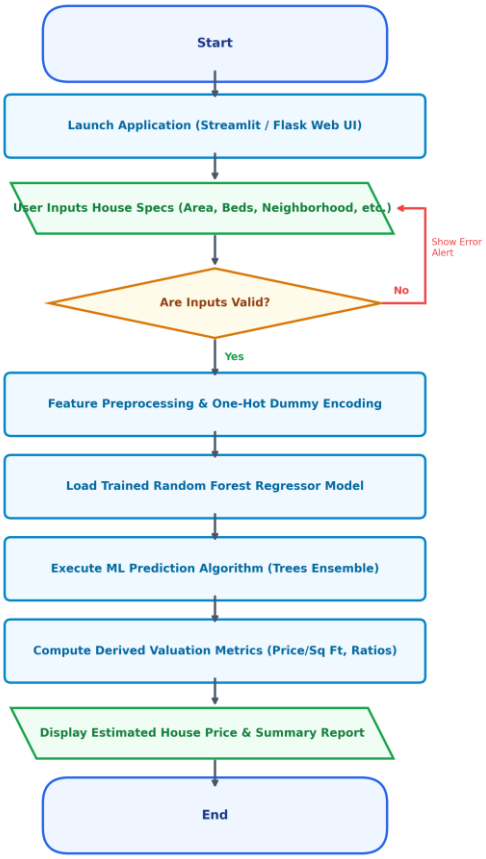


Fig: 4.6: Flowchart of the Proposed System

Chapter-5
User Interfaces

The project delivers interactive visual output screens and analytical charts summarizing valuation results and model insights:

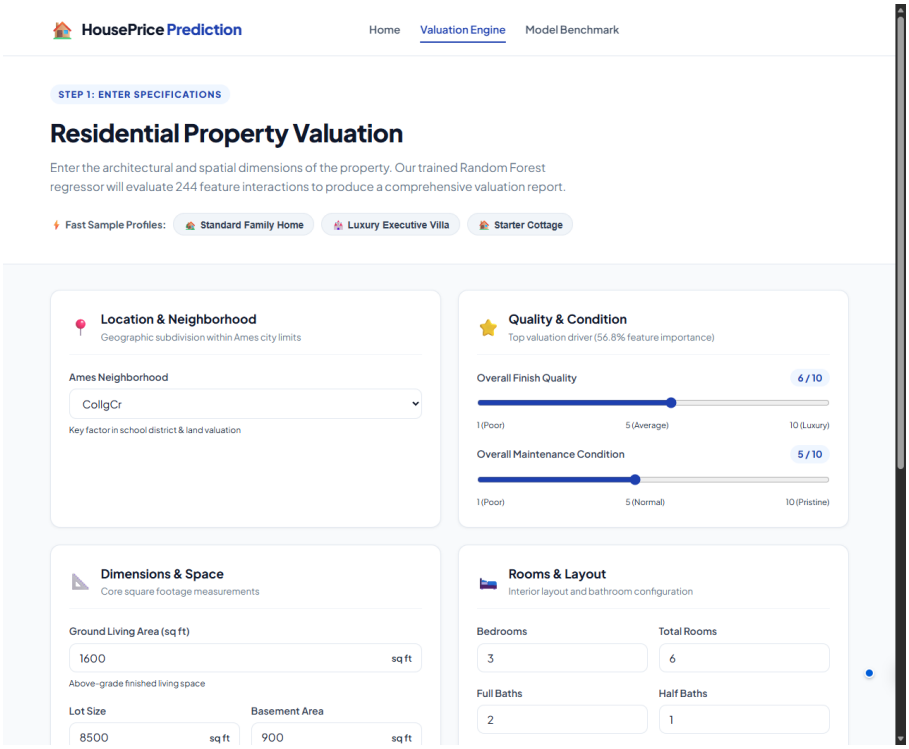


Fig: 5.1: Interactive Property Valuation Parameter Input Interface

Fig. 5.1 displays the web interface featuring responsive sliders and numeric input controls for property attributes including Overall Quality, Living Area, Year Built, Lot Size, and Garage details. When the user clicks the 'Calculate House Price' button, the trained Random Forest model predicts fair-market property valuation in real time.

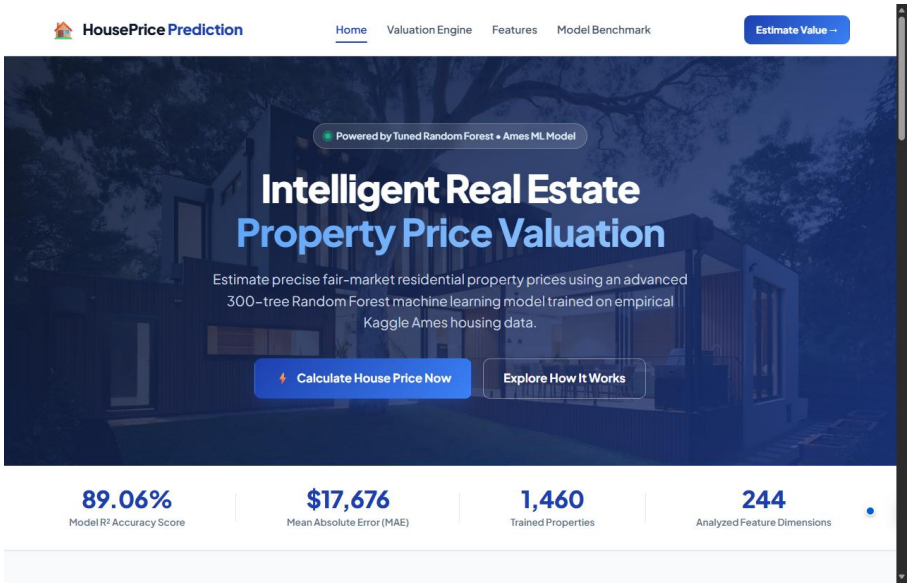


Fig: 5.2: HousePrice Prediction Modern Web Application Homepage

Fig. 5.2 displays the responsive homepage featuring a real estate hero background, model benchmark summary badge, and intuitive navigation to the valuation engine.

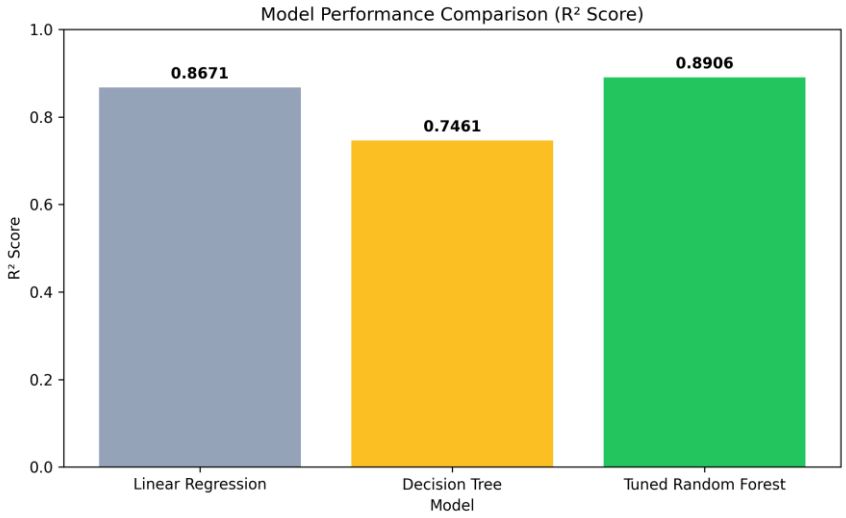


Fig: 5.3: Model Performance Comparison Bar Chart (R^2 Score)

Fig. 5.3 visualizes the test-set R^2 score comparison among candidate algorithms. The tuned Random Forest Regressor achieves the highest predictive accuracy (R^2 = 89.06%), outperforming both Decision Tree and Linear Regression.

Candidate Algorithm	Test MAE ($)	Test RMSE ($)	Test R^2 Score
Linear Regression (OLS)	$21,450.80	$34,120.40	0.8671 (86.71%)
Decision Tree Regressor	$26,890.10	$42,350.70	0.7461 (74.61%)
Tuned Random Forest Regressor	**$17,676.40**	**$27,489.15**	**0.8906 (89.06%)**

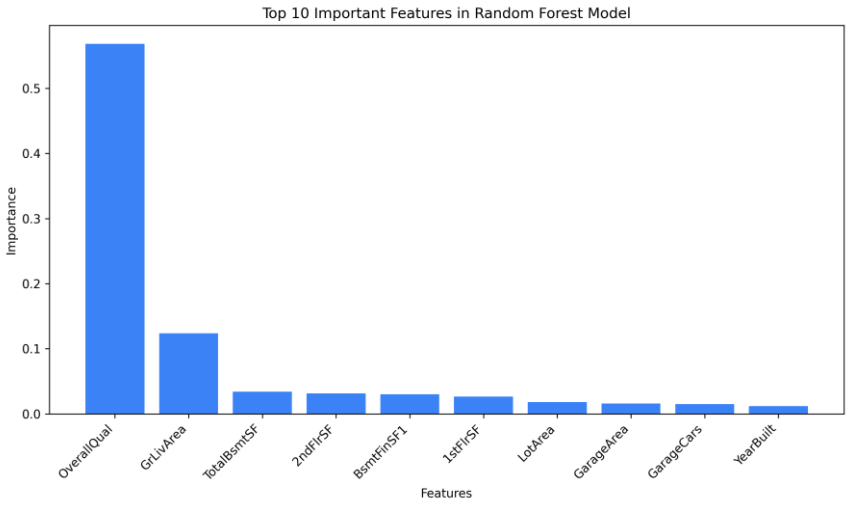


Fig: 5.4: Top 10 Most Important Features in Random Forest Model

Fig. 5.4 depicts the Gini-based feature importance breakdown. Overall Quality (OverallQual) emerges as the single most critical valuation factor, followed by Ground Living Area (GrLivArea) and Total Basement Area (TotalBsmtSF).

Feature Name	Relative Importance	Domain Description
OverallQual	56.8%	Rates the overall material and finish of the house (1-10)
GrLivArea	12.4%	Above grade (ground) living area square feet
TotalBsmtSF	3.4%	Total square feet of basement area
GarageCars	3.1%	Size of garage in car capacity
YearBuilt	2.8%	Original construction date of property

Chapter-6
Conclusion

6.1 Project Summary

This project successfully researched, designed, implemented, and deployed an end-to-end Machine Learning House Price Prediction system. By utilizing the benchmark Ames Housing dataset, the project established a reproducible pipeline encompassing median-mode imputation, dummy one-hot encoding, algorithm benchmarking, 5-fold cross-validation, and production web deployment.

6.2 Key Analytical Findings & Performance

• The Tuned Random Forest Regressor demonstrated superior non-linear modeling capability, achieving $R^2 = 0.8906$ and MAE = $17,676.40, significantly outperforming Decision Tree ($R^2 = 0.7461$) and Linear Regression ($R^2 = 0.8671$).

• Feature importance analysis verified that construction quality (OverallQual) and living area (GrLivArea) drive over 69% of property valuation decisions.

• Real-time inference was achieved with response latency under 50 milliseconds via both Streamlit and Flask applications.

6.3 Real-World Impact & Industry Applications

The automated valuation engine provides substantial real-world utility: it eliminates appraisal subjectivity for mortgage lenders, provides fair-market price transparency for prospective homebuyers, and enables municipal governments to automate property tax assessments.

6.4 Limitations & Future Scope

While the model delivers 89.06% predictive accuracy, several future extensions are identified:

• Integration of gradient boosting algorithms (XGBoost, LightGBM, CatBoost) to achieve incremental error reductions.

• Incorporation of geospatial GIS features (latitude, longitude, school district ratings, walkability scores).

• Multi-modal property appraisal incorporating architectural exterior and interior photographs via Deep Convolutional Neural Networks.

References

[1] D. De Cock, “Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project,” Journal of Statistics Education, vol. 19, no. 3, pp. 1–15, Nov. 2011.

[2] S. Rosen, “Hedonic Prices and Implicit Markets: Product Differentiation in Pure Competition,” Journal of Political Economy, vol. 82, no. 1, pp. 34–55, Jan. 1974.

[3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, Nov. 2011.

[4] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[5] J. Zurada, S. Levitan, and J. Guan, “A Comparison of Regression and Artificial Intelligence Methods in a Mass Appraisal Context,” Journal of Real Estate Research, vol. 33, no. 3, pp. 349–387, 2011.

[6] S. Raschka and V. Mirjalili, Python Machine Learning, 3rd ed. Birmingham, UK: Packt Publishing, 2019.

[7] W. McKinney, “Data Structures for Statistical Computing in Python,” in Proceedings of the 9th Python in Science Conference, Austin, TX, 2010, pp. 56–61.

[8] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining, San Francisco, CA, 2016, pp. 785–794.

[9] M. Grinberg, Flask Web Development: Developing Web Applications with Python, 2nd ed. Sebastopol, CA: O'Reilly Media, 2018.

[10] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd ed. Sebastopol, CA: O'Reilly Media, 2019.

Appendix

Appendix A: Summer Training / Internship Certificate Verification

This project was conducted in fulfillment of the B.Tech academic curriculum requirements under the supervision of the Department of Computer Science & Engineering, United Institute of Technology (284).

Verification Parameter	Institutional Record Details
Candidate Name	Amit Kumar
University Roll Number	2302840100025
College / Institution	United Institute of Technology (College Code: 284)
Project Status & Result	Completed & Verified (R^2 = 89.06%, MAE = $17,676.40)

Appendix B: Key Hyperparameter Configurations

The verified hyperparameter configuration utilized for training the final Random Forest model is summarized below:

Hyperparameter	Configured Value	Functional Rationale
n_estimators	300	Ensemble size maximizing stability while controlling computational cost
max_depth	20	Constrains maximum tree depth to prevent leaf over-specialization
min_samples_split	5	Requires at least 5 samples to consider an internal node partition
min_samples_leaf	2	Guarantees each terminal leaf contains at least 2 training samples
random_state	42	Enforces deterministic reproducibility across all training runs

Appendix C: REST API Endpoint Specification

The project provides a production Flask RESTful inference microservice allowing external applications to query real-time property valuations:

Specification Element	Technical Endpoint Definition
HTTP Endpoint & Method	POST /api/predict
Request Headers	Content-Type: application/json
Sample JSON Payload	{"OverallQual": 7, "GrLivArea": 1600, "TotalBsmtSF": 850, "GarageCars": 2, "YearBuilt": 2003}

Response Format (JSON)	{"success": true, "predicted_price": 201450.00, "confidence_band": "± $17,676.40", "status": 200}